ComicCraft – Defining the Architecture of the Application

1. Introduction

ComicCraft needs an architecture that connects the web interface, FastAPI backend, Gemini models, image generation and PDF creation. A modular architecture makes the system easier to develop and maintain.

2. High-Level Architecture

The system can follow a layered design: User → Web Interface → FastAPI Backend → Gemini and Image Generation Services → Comic Assembly → Preview/PDF.

- Separate responsibilities between modules.
- Use structured request and response formats.
- Keep AI services replaceable.
- Plan for long-running image generation.

3. Presentation Layer

The presentation layer collects user input, validates basic fields, sends requests to the backend, displays progress and renders the generated comic.

4. FastAPI Backend Layer

FastAPI acts as the central coordinator. It defines API routes, validates requests, calls AI services, manages generated files and returns structured results to the frontend.

- Separate responsibilities between modules.
- Use structured request and response formats.
- Keep AI services replaceable.
- Plan for long-running image generation.

5. Gemini AI Layer

Gemini can generate story structure, narration, dialogue, character information and detailed panel descriptions. Structured prompts help produce predictable results.

6. Image Generation Layer

Stable Diffusion with Hugging Face Diffusers can transform panel descriptions into comic-style illustrations. The service creates and stores images for individual panels.

- Separate responsibilities between modules.
- Use structured request and response formats.
- Keep AI services replaceable.
- Plan for long-running image generation.

7. Comic Assembly and PDF Layer

The application combines each image with its narration and dialogue in the correct order. A PDF component then creates a readable downloadable comic.

8. Data Flow

The sequence is: receive input, validate data, generate story, parse panels, create image prompts, generate illustrations, assemble panels, return preview and generate PDF when requested.

- Separate responsibilities between modules.
- Use structured request and response formats.
- Keep AI services replaceable.
- Plan for long-running image generation.

9. Suggested Project Structure

A useful structure separates app/main.py, routes, services, models, templates, static CSS/JavaScript, generated files, requirements.txt and README.md. This separation improves maintainability.

10. Security and Reliability

API keys should be stored in environment variables. Input must be validated. External service failures and timeouts should be handled without exposing internal errors.

- Separate responsibilities between modules.
- Use structured request and response formats.
- Keep AI services replaceable.
- Plan for long-running image generation.

11. Scalability

For larger usage, long-running image generation can be moved to background workers or a task queue. Generated files can also be stored in cloud storage.

12. Testing Architecture

Each module can be tested separately: UI requests, FastAPI endpoints, Gemini response parsing, image generation, comic assembly and PDF creation.

- Separate responsibilities between modules.
- Use structured request and response formats.
- Keep AI services replaceable.
- Plan for long-running image generation.

13. Future Enhancements

Possible improvements include background processing, cloud storage, caching, additional AI models, user accounts, saved comics and separate production services.

14. Conclusion

The architecture creates a clear path from user prompt to AI-generated story, illustrations, comic preview and PDF. Modular services allow individual components to be upgraded without redesigning the entire application.

- Separate responsibilities between modules.
- Use structured request and response formats.
- Keep AI services replaceable.
- Plan for long-running image generation.